Security Review Report

db-backend-architecture-anava — Backend Codebase (FastAPI / PostgreSQL)

Scope	Full backend/app tree (no active branch diff — reviewed as clean HEAD state)
Repository	db-backend-architecture-anava
Date	2026-08-25
Method	Automated multi-agent review — identification pass followed by independent per-finding verification pass with source-code confirmation and false-positive filtering
Findings	9 confirmed (7 High, 2 Medium) — all ≥ 8/10 confidence after verification

> **Root cause underlying most findings:** the application's PostgreSQL connection role has `rolbypassrls = TRUE` (confirmed in `config.py`, `db.py`, `scoping.py`, and `patients/router.py`) — a Postgres superuser/BYPASSRLS role bypasses Row-Level Security policies unconditionally, regardless of `FORCE ROW LEVEL SECURITY` on the table. This means the RLS policies defined in `SQL/v1/17_rls_policies.sql` are effectively dead code in the deployed environment. **Application-layer checks (`assert_clinic_scope`) are the only real tenant boundary** — and across 9 endpoints, that check is missing on the "read one by ID" or "mutate" path while present on the sibling "list" path.

#	Title	Location	Severity	Category	Conf.
1	Cross-clinic patient list leak	`patients/router.py:47`	**High**	authz_bypass	9/10
2	Cross-clinic patient GET-by-ID	`patients/router.py:72`	**High**	authz_bypass	9/10
3	Cross-clinic staff PII leak	`staff/router.py:51`	**Medium**	authz_bypass	9/10
4	Staff-assignments cross-clinic CRUD	`admin/router.py:279`	**High**	authz_bypass	9/10
5	Cross-clinic protocol-instance GET	`treatment_protocols/router.py:323`	**High**	authz_bypass	9/10
6	Clinical protocol-requests — no scoping	`clinical/router.py:25`	**High**	authz_bypass	9/10
7	PRS/anamnesis cross-clinic read+write	`prs/router.py:96`	**High**	authz_bypass	9/10
8	Path traversal in file storage	`files/router.py:68`	**Medium**	path_traversal	9/10

9	Cross-clinic appointment GET	`scheduling/router.py:178`	**Medium**	authz_bypass	9/10

Vuln 1 — Cross-clinic patient list leak via missing role scope

HIGH **authz_bypass** **confidence 9/10**

Location

`backend/app/modules/patients/router.py:47`

Description

`GET /patients` (`list_patients`) auto-forces `clinic_id` to the caller's own clinic only when `ctx.role in ("clinic_admin", "receptionist")`. Doctor and clinical_assistant roles fall through with `clinic_id` left as whatever the caller passed (or `None`). `PatientService.list()` → `PatientRepository.list()` only adds a `pt.primary_clinic_id = :clinic_id` clause when `clinic_id` is truthy, so an unscoped call returns every patient in the system regardless of clinic.

Exploit Scenario

Authenticate as any doctor or clinical_assistant (valid staff credentials at Clinic A suffice). Call `GET /api/v1/patients` with no `clinic_id` query parameter. The role check excludes doctor/CA from the auto-scope branch, so the full patient roster across every clinic is returned — name, DOB, contact info, MRN, primary_clinic_id, assigned doctor, registration/approval status — including patients the caller has no clinical relationship with.

Recommendation

Extend the auto-scoping branch to cover every clinic-pinned staff role: `if ctx.role in ("clinic_admin", "receptionist", "doctor", "clinical_assistant"): clinic_id = UUID(ctx.clinic_id)`. Only super_admin/regional_admin should be allowed to omit clinic_id.

Vuln 2 — Cross-clinic patient record access via GET by ID

HIGH authz_bypass confidence 9/10

Location

`backend/app/modules/patients/router.py:72`

Description

`GET /patients/{patient_id}` (`get_patient`) calls only `assert_patient_self(ctx, db, patient_id)`, which is a documented no-op for every role except `"patient"`. There is no `assert_clinic_scope` call in this handler, unlike `update_patient` / `delete_patient` / `decide_patient_approval` / `allocate_doctor` in the same file, which all fetch the patient first and then call `assert_clinic_scope`.

Exploit Scenario

Any staff role at Clinic A obtains a `patient_id` belonging to Clinic B (trivially available via the Vuln 1 leak). Calling `GET /api/v1/patients/{that_patient_id}` returns the full record — guardian details, emergency contacts, allocated doctor, registration status, address — with no clinic check.

Recommendation

After fetching the patient, call `assert_clinic_scope(ctx, db, patient["primary_clinic_id"])` for staff roles, mirroring the pattern already used on the PATCH/DELETE/approval/allocate-doctor routes in this file.

Vuln 3 — Cross-clinic staff PII exposure via GET-by-ID endpoints

MEDIUM authz_bypass confidence 9/10

Location

`backend/app/modules/staff/router.py:51`

Description

`GET /doctors/{id}`, `GET /clinical-assistants/{id}`, and `GET /receptionists/{id}` are gated only by `require_role(*_STAFF_READ_ROLES)` with no `assert_clinic_scope` call — inconsistent with the `update_*` / `delete_*` handlers for the same resources in the same file, which do call `assert_clinic_scope(ctx, db, existing["clinic_id"])`.

Exploit Scenario

Any staff role at Clinic A obtains a staff ID belonging to Clinic B (enumerable/learnable via other list/detail flows in the app). Calling the corresponding GET-by-ID endpoint discloses full staff PII (name, email, phone, specialization, clinic assignment) for a clinic the caller has no relationship with.

Recommendation

Add `await assert_clinic_scope(ctx, db, result["clinic_id"])` after the fetch in all three GET-by-ID handlers, mirroring the check already present in their PATCH/DELETE siblings.

Vuln 4 — Clinic_admin can view/add/remove staff assignments at any clinic

HIGH authz_bypass confidence 9/10

Location

`backend/app/modules/admin/router.py:279`

Description

`GET/POST /clinics/{clinic_id}/staff-assignments` and `PATCH /staff-assignments/{assignment_id}` have no `assert_clinic_scope` call anywhere, unlike `update_clinic` in the same file. `StaffAssignmentService` methods pass `clinic_id` / `assignment_id` straight through to the repository with no ownership check; `remove` filters only by `assignment_id`.

Exploit Scenario

A clinic_admin at Clinic A calls `GET /api/v1/clinics/{clinic_B_id}/staff-assignments` to list Clinic B's staff roster, `POST`s a new assignment at Clinic B, and `PATCH`es/removes an existing Clinic B assignment — none of it rejected, since role checks never compare clinic ownership.

Recommendation

Call `assert_clinic_scope(ctx, db, clinic_id)` before delegating in list/add; for remove, look up the assignment's clinic_id first and check before allowing removal.

Vuln 5 — Cross-clinic treatment-episode access via protocol-instance GET

HIGH authz_bypass confidence 9/10

Location

`backend/app/modules/treatment_protocols/router.py:323`

Description

`GET /protocol-instances/{instance_id}` calls `ProtocolInstanceService.get_or_404` (no clinic filtering) then only `assert_owns_profile`, a no-op for non-patient roles — inconsistent with `ProtocolService.get_or_404` and `ProtocolInstanceService.list()`, both of which correctly scope by clinic.

Exploit Scenario

Staff at Clinic A obtains an `instance_id` for a Clinic B treatment episode (via patient leaks above, or via the instance_id's normal appearance as a foreign key in scheduling/order payloads) and calls the GET route, receiving full clinical episode data (instance type, status, doctor/CA assignment, notes) with no clinic check.

Recommendation

Add `assert_clinic_scope(ctx, session, row["clinic_id"])` for non-patient callers, mirroring the pattern already used elsewhere in this service file.

Vuln 6 — Clinical protocol-request module has no clinic scoping anywhere

HIGH authz_bypass confidence 9/10

Location

`backend/app/modules/clinical/router.py:25`

Description

`GET /assessment-protocol-requests` (list), `GET .../{request_id}` (get), and `PATCH .../{request_id}/decision` (decide) are gated only by role membership. Neither the router, `ProtocolRequestService`, nor `ProtocolRequestRepository` ever filters or checks `clinic_id` — confirmed by grep across the whole module.

Exploit Scenario

A clinical_assistant or doctor at Clinic A lists protocol requests system-wide (no clinic filter on list()), reads any specific request, and — as a doctor at any clinic — approves/rejects a Clinic B request via the decision endpoint, which mutates state (auto-creates `patient_scale_assignments`) for a patient the doctor has no relationship with.

Recommendation

Filter `list()` by `ctx.clinic_id` for clinic-pinned staff roles; add `assert_clinic_scope` in `get()` and before `decide()` proceeds.

Vuln 7 — PRS and anamnesis clinical data readable/writable across clinic boundaries

HIGH authz_bypass confidence 9/10

Location

`backend/app/modules/prs/router.py:96`

Description

`get_assessment`, `list_responses`, `submit_responses`, `get_results` (PRS) and `get_anamnesis_responses` / `submit_anamnesis_responses` (anamnesis) all call only `assert_owns_profile`, a no-op for staff. No clinic-scoping logic exists anywhere in either service.

Exploit Scenario

The exploit chain is self-contained: staff at Clinic A supply an arbitrary `patient_id` to `list_patient_prs_instances` / `get_current_anamnesis` (also unscoped) to obtain instance/anamnesis IDs for a Clinic B patient, then read/modify full PRS scale scores and anamnesis psychiatric intake responses for that unrelated-clinic patient.

Recommendation

Resolve each instance's/assessment's clinic via its patient and call `assert_clinic_scope` for non-patient callers in every one of these handlers, before allowing read or write access.

Vuln 8 — Path traversal in local file storage download/upload routes

MEDIUM path_traversal confidence 9/10

Location

`backend/app/modules/files/router.py:68`

Description

`GET /files/download/{s3_key:path}` and `PUT /files/upload/{s3_key:path}` use Starlette's `:path` converter (regex `.*`, no normalization). The only defense, `_assert_own_key()`, does an unanchored substring check — `f"/patients/{own_patient_id}/" not in s3_key` — trivially satisfied by embedding that substring anywhere in a longer traversal path. The filesystem path is then built via raw string concatenation with no containment check. `file_storage_mode` defaults to `"local"`, which is the live deployment mode, not test-only.

Exploit Scenario

Any authenticated user (patient or staff) embeds `/patients/{own_patient_id}/` plus `../` traversal segments in the `s3_key` path parameter, escaping the intended storage root and reading (or, via the PUT route, writing) arbitrary files on the host filesystem within the web process's permissions.

Recommendation

Normalize and validate containment: resolve `(Path(local_root) / s3_key).resolve()` and reject unless `is_relative_to(Path(local_root).resolve())`. Anchor `_assert_own_key` with an exact prefix `startswith` check instead of an unanchored substring test.

Vuln 9 — Cross-clinic appointment access for clinical_assistant/receptionist/clinic_admin

MEDIUM authz_bypass confidence 9/10 (corrected)

Location

`backend/app/modules/scheduling/router.py:178`

Description

`GET /appointments/{appointment_id}` and `GET /appointments/{appointment_id}/audit-log` only raise a permission error when `ctx.role == "patient"` or `"doctor"` and the ID doesn't match the caller. There is no check at all for clinical_assistant/receptionist/clinic_admin, inconsistent with `AppointmentService.list()`, which correctly forces `clinic_id = ctx.clinic_id` for exactly these three roles.

Exploit Scenario

A clinical_assistant or receptionist at Clinic A obtains a Clinic B `appointment_id` (via cross-clinic patient/instance enumeration from the other findings) and calls the GET route, receiving the full appointment (including `reason` / `patient_complaint`) and its audit-log history with no clinic check.

Recommendation

Replace the two explicit role checks with the same clinic_id comparison `AppointmentService.list()` already applies.

> **Verification note:** an initial verification pass rated this a false positive, reasoning that PostgreSQL `FORCE ROW LEVEL SECURITY` on the `appointments` table would block the leak. That conclusion was incorrect: this codebase's connecting DB role has `rolbypassrls = TRUE` (confirmed in 4 separate places in the code — `config.py`, `db.py`, `scoping.py`, `patients/router.py`), and Postgres roles with the BYPASSRLS attribute bypass Row-Level Security unconditionally — `FORCE ROW LEVEL SECURITY` only affects the table owner, not a BYPASSRLS role. The finding was re-confirmed and is included above.

Bottom line: a systemic pattern runs across nearly every module — the "list" and "PATCH/DELETE" endpoints correctly call `assert_clinic_scope`, but the sibling "GET by ID" or "decide/mutate" endpoint for the same resource does not. This suggests the omission is structural (a missed step in a shared scaffolding pattern) rather than isolated per-endpoint mistakes. A codebase-wide sweep for this exact pattern is recommended, since modules not deep-dived in this pass (inventory, notifications, store) may carry the same gap.

Generated via automated multi-agent security review: an identification pass across the full backend/app tree, followed by independent per-finding verification against source code with false-positive filtering (confidence ≥ 8/10 required to retain a finding). All 9 findings confirmed by direct code inspection, not by dynamic testing or exploitation.